TI-2400 Processamento de Linguagem Natural

ANTONIO CARVALHO - TREINAMENTOS

Visualização de Dados com Matplotlib

O que é visualização de dados ?

Data visualization (Visualização de dados) é o uso de representações gráficas para explorar, compreender e comunicar informações presentes em conjuntos de dados, objetivando explorar, comparar e comunicar dados. Alguns tipos de ilustrações facilitam este processo como gráficos, mapas, figuras e fluxos.

Em processamento de linguagem natural usaremos ***data visualization*** para:

- verificar palavras mais frequentes
- distribuição do tamanho dos textos
- frequência de classes
- comparação entre documentos
- resultados de modelos
- matriz de confusão

A ferramenta Matplotlib pemite a exibição de gráficos e figuras para-auxiliar neste processo.

Normalmente é essencial analisar grandes volumes de dados e tomar decisões orientadas aos dados.

O importante é transmitir a compreensão, portanto é essencial converter os dados complexos em uma maneira de fácil compreensão

Matplotlib

- **Matplotlib** é uma das ferramentas mais poderosas em Python para visualização de dados. É uma biblioteca que permite a criação de gráficos em 2-D e 3-D
- Para instalar o **matplotlib** é preciso executar:

```
pip install matplotlib
```

Ou

```
python –m pip install matplotlib
```

- Uma vez instalado basta importá-lo para seu código usando

```
import matplotlib.pyplot as plt
```

Matplotlib

Há duas maneiras de trabalhar com o Matplotlib:

- Interface **pyplot**
- Interface **orientada a objetos**

Interface pyplot

```
plt.plot(x, y)
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

Interface Orientada a Objetos

```
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_xlabel("X")
ax.set_ylabel("Y")
plt.show()
```

Matplotlib

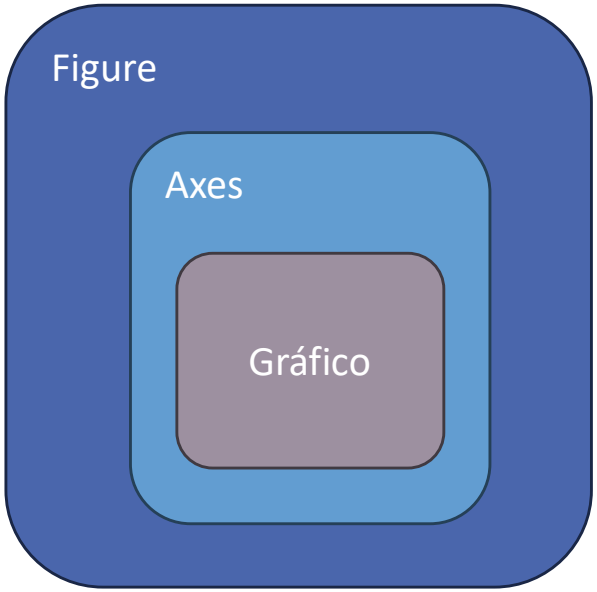
Conceitos Figure, Axes e Axis

Uma visualização no Matplotlib normalmente possui:

- **Figure**, representa a figura completa.
- **Axes**, representa uma área onde um gráfico é desenhado.
- **Axis**, representa os eixos do gráfico: eixo X eixo Y

Podemos criar uma figura e uma área de gráfico utilizando:

```
fig, ax = plt.subplots()
ax.plot(
      [1, 2, 3],
      [10, 20, 15]
)
plt.show()
```



Matplotlib - subplots

O Matplotlib permite colocar vários gráficos dentro da mesma Figure `fig, axes = plt.subplots( <linhas>, <colunas>)`

Podemos utilizar **subplots()** para comparar as palavras mais frequentes antes e depois da remoção de **stopwords**.

```
import matplotlib.pyplot as plt
fig, axes = plt.subplots(1, 2)
axes[0].bar(palavras_antes, frequencias_antes)
axes[0].set_title("Antes das Stopwords")
axes[1].bar(palavras_depois, frequencias_depois)
axes[1].set_title("Depois das stopwords")
plt.tight_layout()
plt.show()
```

Matplotlib

Configurar a visibilidade e o Estilo das linhas de grade (grid) em um gráfico.

Assinatura:

```
plt.grid(visible=None, which='major', axis='both', **kwargs)
```

visible: Define se a grade aparece ou não. Aceita True (ativar) ou False (desativar). Se você apenas chamar plt.grid(), ele alterna o estado atual (liga se estiver desligada).

axis: Define em qual eixo a grade será aplicada.

- **'both'** (padrão): Aplica nas linhas horizontais e verticais.
- **'x'**: Aplica apenas nas linhas verticais (eixo X).
- **'y'**: Aplica apenas nas linhas horizontais (eixo Y).

Matplotlib

Configurar a visibilidade e o Estilo das linhas de grade (grid) em um gráfico.

which: Define para quais marcações a grade será desenhada.

- **'major'** (padrão): Apenas nas marcações principais.
- **'minor'**: Apenas nas marcações secundárias.
- **'both'**: Em ambas.

****kwargs**: Argumentos extras para mudar a aparência estética das linhas

- **color ou c**: Cor da linha (ex: 'red', '#00FF00', 'gray').
- **linestyle ou ls**: Estilo da linha (ex: '-' contínua, '--' tracejada, ':' pontilhada).
- **linewidth ou lw**: Espessura da linha (ex: 0.5, 1.5).
- **alpha**: Transparência da linha (de 0.0 totalmente invisível a 1.0 totalmente opaca).

Matplotlib

Ajustar automaticamente o espaçameno entre os subplots (gráficos) da figura.

Assinatura:

```
plt.tight_layout(pad=1.08, h_pad=None, w_pad=None, rect=None)
```

pad: Define o espaçamento (margem) entre a borda da figura e os subplots. O padrão é 1.08.

w_pad: Espaçamento horizontal específico entre subplots vizinhos.

h_pad: Espaçamento vertical específico entre subplots vizinhos.

rect: Uma tupla (left, bottom, right, top) em coordenadas normalizadas (de 0 a 1) que define uma "caixa" onde os subplots devem ficar. Muito útil se você quiser deixar um espaço reservado no topo para um título geral (plt.suptitle)

Matplotlib

Cada função do **`pyplot`** serve para fazer mudanças na figura :

- exemplos
  - Criar figura,
  - Criar uma area de gráficos na figura,
  - Desenhar uma linha área de gráficos,
  - Adicionar labels, e títulos.

Ao desenhar algo no matplotlib é preciso se atentar para definir o tipo de gráfico (bar, pie, line, etc)

Gráfico de linhas

```
import matplotlib.pyplot as plt

# cria dados para serem mostrados
x_values = [0, 1, 2, 3, 4, 5 ]
y_values = [0, 1, 4, 9, 16,25]

# plt.plot() conecta os pontos informados
# utilizando linhas por padrão.
plt.plot(x_values, y_values)

# mostra o gráfico
plt.show()
```

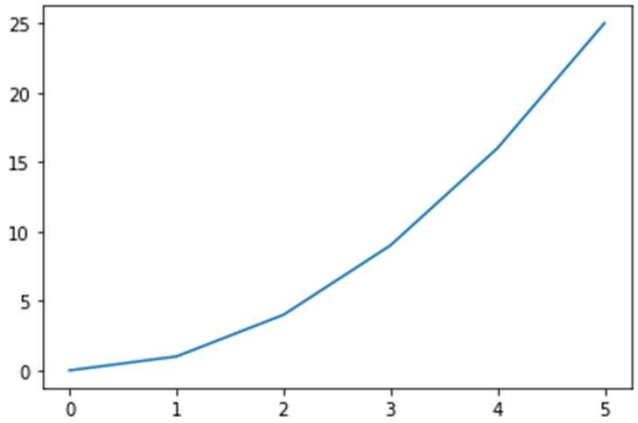


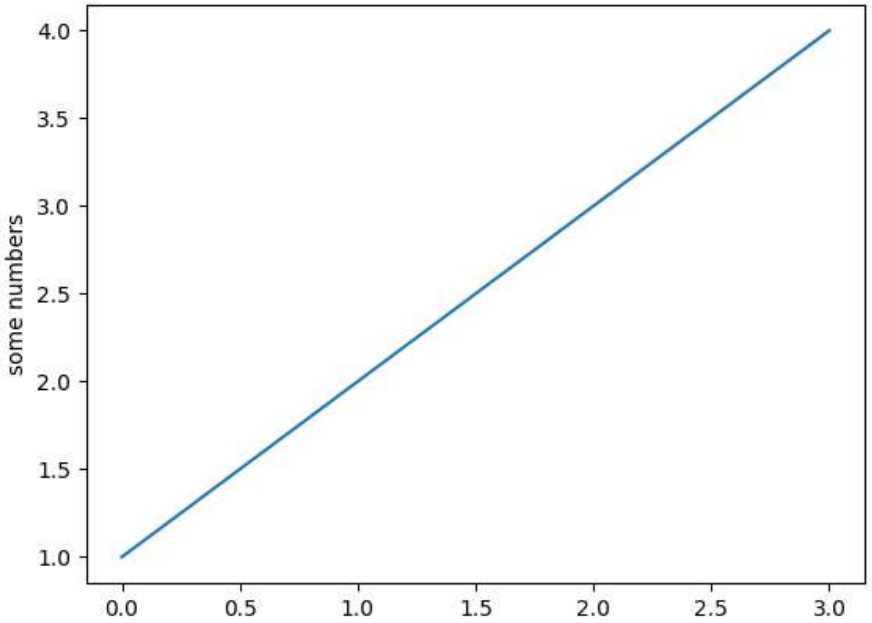
Gráfico de linhas

Nota: caso você informe apenas uma lista para o commando `plot()`, ele assumira esta lista como sendo o **eixo y**, e assumirá valores para o **eixo x**

Os valores gerados automaticamente no **eixo x** começarão do **0**

Os numeros gerados para o eixo x no exemplo a seguir são: [0, 1, 2, 3]

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3, 4])
# x gerado automaticamente:[0, 1, 2, 3]
plt.ylabel('some numbers')
plt.show()
```



Pyplot

- `text()` : **adiciona** um texto em uma posição arbitrária

`xlabel()`: **adiciona** texto para o **eixo x**

`ylabel()`: **adiciona** texto para o **eixo y**

`title()` : **adiciona** titulo para o grafico

`clf()` : **remove** todos os desenhos da área de gráfico

`savefig()`: **salva** o resultado para um arquivo

`legend()` : **mostra** a legenda no gráfico

Muitas operações disponíveis em `pyplot` possuem operações equivalentes em objetos `Axes`, embora os nomes dos métodos possam ser diferentes.

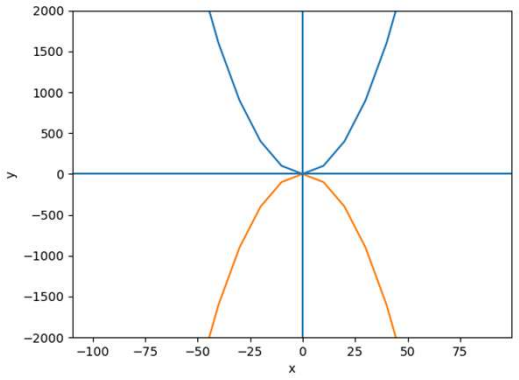
Pyplot

```
import matplotlib.pyplot as plt
y1 =[]
y2 =[]
x = range(-100,100,10)
for i in x: y1.append(i**2)
for i in x: y2.append(-i**2)

plt.plot(x, y1)
plt.plot(x, y2)
plt.xlabel("x")
plt.ylabel("y")
plt.ylim(-2000, 2000)
plt.axhline(0) # horizontal line
plt.axvline(0) # vertical line

plt.savefig("quad.png")

plt.show()
```



Modifica a figura incrementalmente

Salva a imagem para um arquivo

Mostra na tela

Exercício - quantidade de palavras por documento

- Considere cinco textos de um conjunto de dados.
- A quantidade de palavras encontrada em cada texto foi:
  - Texto 1 → 120
  - Texto 2 → 180
  - Texto 3 → 150
  - Texto 4 → 210
  - Texto 5 → 240
- Crie um gráfico de linhas mostrando a quantidade de palavras encontrada em cada documento.
- O gráfico deverá possuir:
  - título
  - nome do eixo X
  - nome do eixo Y
  - marcadores nos pontos

Exercício - Comparação entre dois conjuntos de textos

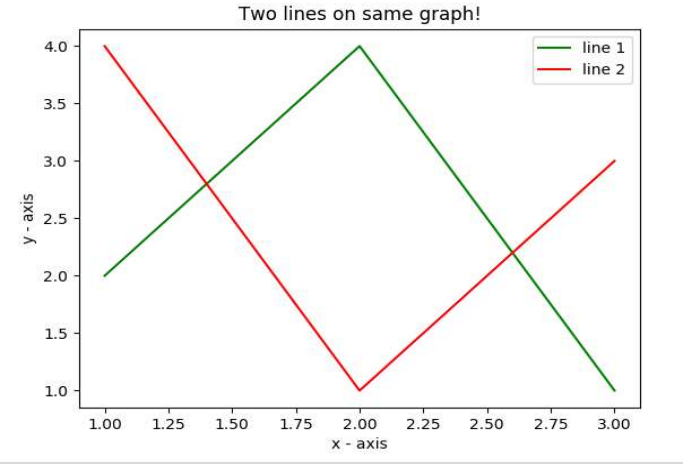
- Foram analisados textos positivos e negativos.
- Número médio de palavras:
- Documento Positivos Negativos
  - 1 120 150
  - 2 140 130
  - 3 170 180
  - 4 160 210
  - 5 190 200
- Construa um gráfico contendo as duas séries e adicione:
  - título
  - labels dos eixos
  - legenda

Exercício – Gráfico com duas séries

- Faça o gráfico ao lado, porém desta vez acione a função `.plot()` uma vez para cada série a ser adicionada ao gráfico.

```
plt.plot(x1, y1, label="line 1")
```

- Ative a legenda utilizando a função `.legend()`



Customização do traçado e dos pontos

```
import matplotlib.pyplot as plt
# valores do eixo x
x = [1, 2, 3, 4, 5, 6]
# valores do eixo y
y = [2, 4, 0, 5, 2, 6]
# desenhando o gráfico
plt.plot(x, y, color='green', linestyle='dashed',
         linewidth = 3, marker='o',
         markerfacecolor='blue', markersize=12)
# definindo as faixas limites de x e y
plt.ylim(0,8)
plt.xlim(1,8)
# colocando label no eixo x
plt.xlabel('x - axis')
# colocando label no eixo y
plt.ylabel('y - axis')
# colocando um título no gráfico
plt.title('Some cool customizations!')
plt.show()
```

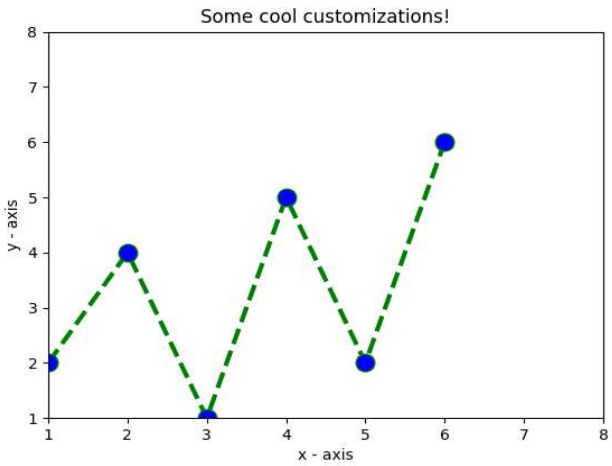


Gráfico de barras

```
import matplotlib.pyplot as plt

#Cria os dados para serem mostrados
palavras = ["dados", "modelo", "texto", "python", "linguagem"]
frequencias = [ 35, 27, 25, 18, 15]
plt.bar(palavras, frequencias, color="green")
plt.title("Palavras mais frequentes")
plt.xlabel("Palavra")
plt.ylabel("Frequência")
plt.show()
```

Utilize `plt.bar()` para mostrar o gráfico como sendo de barras no lugar do `plt.plot()`

Gráfico de barras

É possível trocar a orientação do gráfico de barras

```
import matplotlib.pyplot as plt

#Cria os dados para serem mostrados
palavras = ["dados", "modelo", "texto", "python", "linguagem"]
frequencias = [ 35, 27, 25, 18, 15]
plt.barh(palavras, frequencias, color="yellowgreen")
plt.title("Palavras mais frequentes")
plt.xlabel("Frequência")
plt.ylabel("Palavra")
plt.show()
```

WordCloud é ótima para exploração visual, já o gráfico de barras é melhor quando queremos comparar quantitativamente frequências.

Histograma

```
import matplotlib.pyplot as plt

# observações individuais
quantidade_palavras = [ 45, 82, 120, 98, 105,    150, 210, 74, 89, 132,
170, 60, 115, 140, 185]

# definindo as faixas e intervalos
intervalo = (0, 250)
bins = 5

# desenhando o histograma
plt.hist( quantidade_palavras, bins=bins, range=intervalo,
                color='green',histtype='bar',
                rwidth=0.8)

plt.xlabel("Quantidade de palavras")
plt.ylabel("Número de documentos")
plt.title("Distribuição do tamanho dos textos")
plt.show()
```

Gráfico de Barras × Histograma

Embora sejam visualmente semelhantes, representam informações diferentes.

Gráfico de barras, compara categorias, exemplo:

- Python 35
- Dados 28
- Texto 20

Histograma, mostra a distribuição de uma variável numérica, exemplo: Quantidade de palavras por documento.

Gráfico de pontos

Scatter plots ajudam a investigar possíveis relações entre duas variáveis quantitativas.

```
import matplotlib.pyplot as plt

# dados criados
palavras = [100, 150, 200, 250, 300]
palavras_unicas = [65, 90, 110, 130, 145]

plt.scatter( palavras, palavras_unicas, s=30, color="blue")
plt.xlabel("Quantidade de palavras")
plt.ylabel("Palavras únicas")
plt.title("Tamanho do texto × Vocabulário")
plt.show()
```

Gráfico de pizza

Gráficos de setores são úteis para mostrar proporções com poucas categorias, mas tornam comparações difíceis quando existem muitas categorias ou valores semelhantes.

```
import matplotlib.pyplot as plt

# Definindo labels
activities = ['eat', 'sleep', 'work', 'play']
# Porção coberta de cada label
slices = [3, 7, 8, 6]
# Cores para cada label
colors = ['r', 'y', 'g', 'b']

# plotting the pie chart
plt.pie( slices, labels = activities,
                  colors=colors, startangle=90,
                  shadow = True,
                  explode = (0, 0, 0.1, 0),
                  radius = 1.2, autopct = '%1.1f%%')
plt.legend()
plt.show()
```

